

Prototipo Zoom

Sistema de Videoconferencia por Sockets

PC4 - SW403U - Programación Orientada a Objetos

Universidad Nacional de Ingeniería

Facultad de Ingeniería Industrial y Sistemas

Integrantes:

Yampufe Pau Héctor Alonso - 20240310I

Romaní Cuyo Gabriel Alberto - 20242045K

Tasayco Portello Luis Antonio - 20240307H

Docente: CISNEROS NAPRAVNIK YAN EDUARDO

26 de junio de 2026

Índice

1. Introducción	3
2. Objetivos	3
3. Tecnologías Utilizadas	3
4. Arquitectura del Sistema	4
5. Base de Datos	4
6. Diagrama Entidad-Relación	5
7. Diagrama de Casos de Uso	6
8. Especificación de Casos de Uso	6
9. Diccionario de Datos	8
10. Diagrama de Secuencia	10
11. Diagrama de Actividades	11
12. Diagrama de Clases	12
13. Descripción del Código	13
13.1. Funcionamiento de las Clases y Componentes	13
13.1.1. Lado del Servidor	13
13.1.2. Lado del Cliente	13
13.1.3. Gestores de Red y Hardware	14
13.2. Aplicación de los Pilares de la POO	14
13.2.1. Encapsulamiento	14
13.2.2. Herencia	15
13.2.3. Polimorfismo	15
13.2.4. Abstracción	16
13.3. Patrones de Diseño Implementados	16
13.3.1. Singleton (Instancia Única)	16

13.3.2. Observer (Observador)	17
13.3.3. Strategy (Estrategia)	17
13.3.4. Facade (Fachada)	17
13.3.5. Command (Comando)	18
13.3.6. Factory Method (Método Fábrica)	18
13.3.7. Template Method (Método Plantilla)	18
14. Protocolos y Normativas Utilizados	18
14.1. Tabla de Mensajes del Protocolo	19
15. Manual de Instalación y Ejecución	20
15.1. Requisitos	20
15.2. Ejecución	20
15.3. Datos de Prueba	20
16. Conclusiones	20

1. Introducción

El presente documento describe el diseño e implementación de un **prototipo académico de videoconferencia** inspirado en Zoom, desarrollado en Python como parte del curso de Programación Orientada a Objetos (PC4).

El sistema emplea una arquitectura **cliente-servidor** basada en **sockets TCP** con un protocolo de mensajes en formato **JSON**. Cuenta con autenticación de usuarios, creación y gestión de salas virtuales, chat en tiempo real, transferencia de archivos y transmisión de video desde la cámara web.

2. Objetivos

- Aplicar los principios de la POO: herencia, encapsulamiento, polimorfismo.
- Implementar una arquitectura cliente-servidor mediante sockets TCP.
- Desarrollar un protocolo de comunicación propio basado en JSON.
- Integrar múltiples funcionalidades: login, salas, chat, archivos y video.
- Utilizar SQLite para la persistencia de información.
- Emplear concurrencia con **threading** para atender múltiples clientes.

3. Tecnologías Utilizadas

- **Lenguaje:** Python 3.10+
- **Interfaz gráfica:** Tkinter
- **Comunicación:** Sockets TCP (RFC 793)
- **Protocolo de mensajes:** JSON sobre TCP (RFC 8259), delimitado por \n
- **Base de datos:** SQLite 3
- **Concurrencia:** threading (modelo de hilos POSIX)
- **Video:** OpenCV 4.x (backend MJPEG) o ffmpeg (V4L2 en Linux)

- **Audio:** PortAudio v19 mediante `sounddevice` (PCM 16 bit, 16 kHz mono)
- **Archivos:** transferencia binaria codificada en Base64 (RFC 4648)
- **Seguridad:** hash SHA-256 para contraseñas (FIPS 180-4)
- **Imágenes:** Pillow (PIL Fork)

4. Arquitectura del Sistema

El sistema sigue un modelo cliente-servidor. El servidor centraliza las conexiones y actúa como intermediario. Cada cliente se conecta mediante TCP y el servidor asigna un hilo independiente por conexión.

5. Base de Datos

La base de datos SQLite contiene las siguientes tablas:

Tabla	Columnas
Usuarios	IdUsuario, Nombres, Correo, PasswordHash, Rol, Activo
Salas	IdSala, CodigoSala, Nombre, IdHost, Estado, FechaCreacion
ParticipantesSala	IdParticipante, IdSala, IdUsuario, Estado, FechaIngreso
SolicitudesSala	IdSolicitud, IdSala, IdUsuario, Estado, FechaSolicitud
Mensajes	IdMensaje, IdSala, IdUsuario, Contenido, FechaEnvio
ArchivosCompartidos	IdArchivo, IdSala, IdUsuario, NombreArchivo, RutaArchivo, Tamano, FechaSubida

Cuadro 1: Tablas de la base de datos

6. Diagrama Entidad-Relación

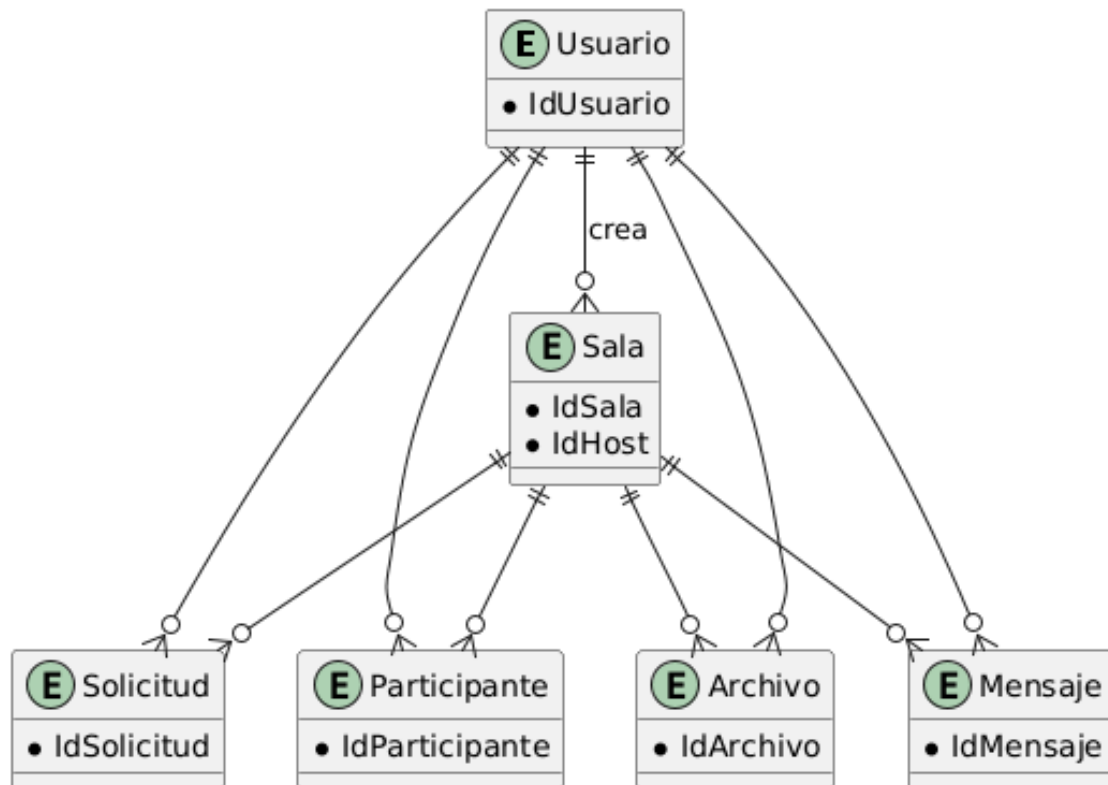


Figura 1: Diagrama Entidad-Relación de la base de datos

7. Diagrama de Casos de Uso

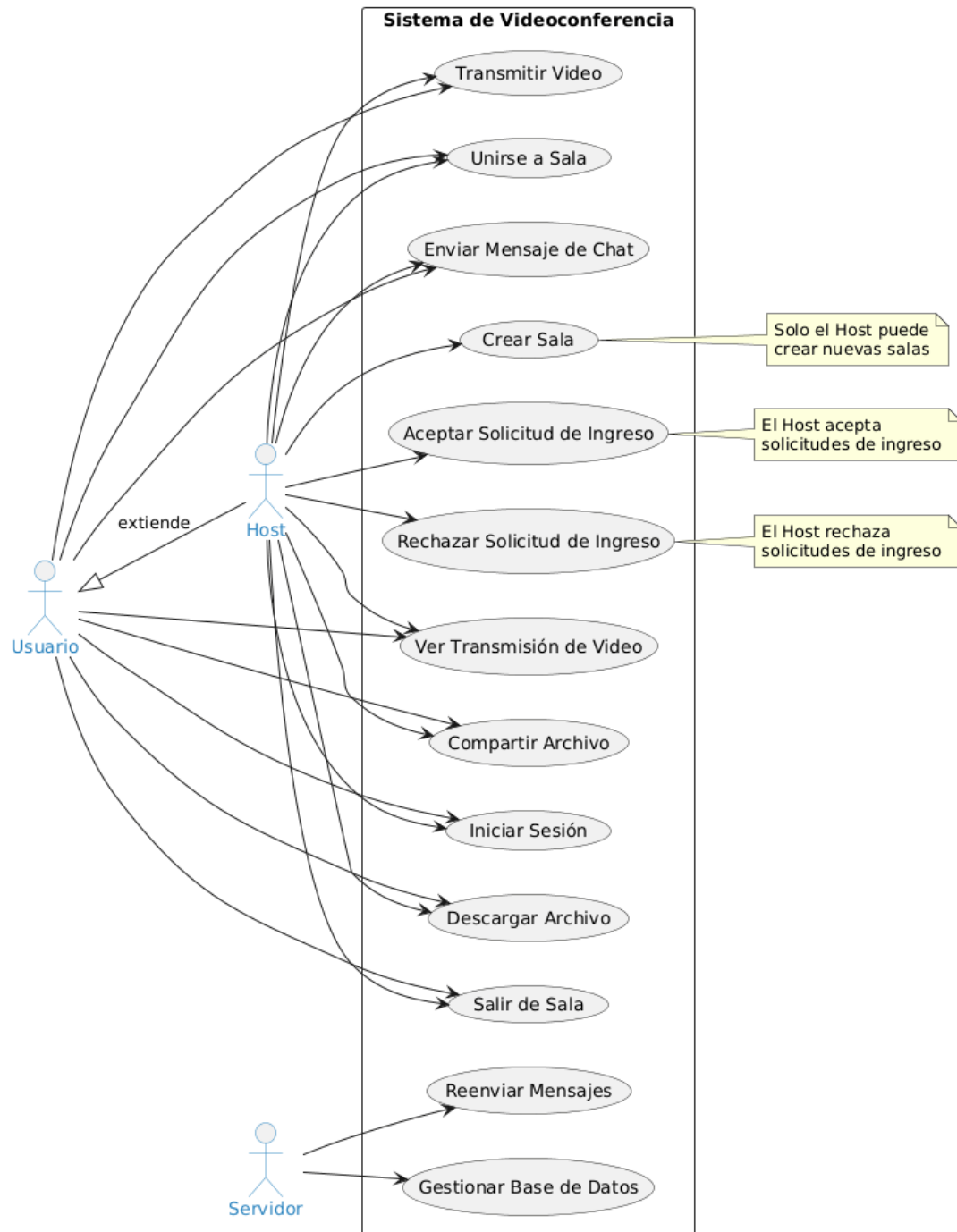


Figura 2: Diagrama de casos de uso del sistema

8. Especificación de Casos de Uso

Se describen a continuación los casos de uso principales del sistema.

CU-01: Iniciar Sesión

Campo	Descripción
Actor	Usuario registrado
Precondición	El servidor está activo y el usuario tiene cuenta en el sistema
Flujo principal	1. El usuario ingresa correo y contraseña. 2. El cliente envía <code>LOGIN_REQUEST</code> al servidor. 3. El servidor valida las credenciales con hash SHA-256. 4. El servidor responde con <code>LOGIN_RESPONSE</code> (<code>status: success</code>). 5. El cliente muestra la pantalla principal.
Flujo alternativo	Si las credenciales son incorrectas, el servidor responde con <code>status: error</code> y se muestra un mensaje al usuario.
Postcondición	El usuario queda autenticado y puede crear o unirse a salas.

Cuadro 2: CU-01: Iniciar Sesión

CU-02: Crear Sala

Campo	Descripción
Actor	Usuario autenticado (rol Host)
Precondición	El usuario ha iniciado sesión exitosamente
Flujo principal	1. El usuario presiona “Crear Sala”. 2. El cliente genera un código alfanumérico de 6 caracteres. 3. El cliente envía <code>CREATE_ROOM</code> con código y nombre. 4. El servidor registra la sala en la BD y asigna al usuario como host. 5. El cliente accede directamente a la sala como administrador.
Flujo alternativo	Si el código ya existe, el servidor retorna error y el cliente intenta con otro código.
Postcondición	La sala queda activa en la base de datos y el host puede gestionar participantes.

Cuadro 3: CU-02: Crear Sala

CU-03: Unirse a Sala

Campo	Descripción
Actor	Usuario autenticado
Precondición	Existe una sala activa con el código ingresado
Flujo principal	1. El usuario ingresa el código de sala y presiona “Unirse”. 2. El cliente envía <code>JOIN_ROOM_REQUEST</code>. 3. El servidor notifica al host con <code>WAITING_ROOM_UPDATE</code>. 4. El host admite al usuario (<code>ADMIT_USER</code>). 5. El servidor notifica al invitado y lo agrega a la sala.
Flujo alternativo	Si el host rechaza (<code>REJECT_USER</code>), el cliente recibe notificación de rechazo y regresa al menú.
Postcondición	El usuario es participante activo de la sala.

Cuadro 4: CU-03: Unirse a Sala

CU-04: Enviar Archivo

Campo	Descripción
Actor	Participante de la sala
Precondición	El usuario está dentro de una sala activa
Flujo principal	1. El usuario presiona “Archivo” y selecciona un fichero. 2. El cliente codifica el archivo en Base64 y lo fragmenta en chunks de 1500 bytes. 3. Envía <code>FILE_START</code>, los <code>FILE_CHUNK</code> y finalmente <code>FILE_END</code>. 4. Los receptores son consultados si desean descargar el archivo. 5. El receptor acepta y el archivo se reconstruye y guarda en <code>descargas/</code>.
Flujo alternativo	Si el receptor rechaza la descarga, el archivo no se guarda en ese cliente.
Postcondición	El archivo queda disponible en el equipo de los receptores que aceptaron.

Cuadro 5: CU-04: Enviar Archivo

9. Diccionario de Datos

Se describe cada atributo relevante de las entidades principales del sistema.

Tabla: Usuarios

Campo	Tipo	Descripción
IdUsuario	INTEGER	Clave primaria autoincrementable. Identificador único del usuario.
Nombres	TEXT	Nombre completo del usuario. No puede ser nulo.
Correo	TEXT	Dirección de correo electrónico. Debe ser única en el sistema.
PasswordHash	TEXT	Hash SHA-256 de la contraseña en hexadecimal. Nunca se almacena texto plano.
Rol	TEXT	Rol del usuario: Host o Usuario . Valor por defecto: Usuario .
Activo	INTEGER	Indicador booleano: 1 = activo, 0 = inactivo (baja lógica).

Cuadro 6: Diccionario de datos – Usuarios

Tabla: Salas

Campo	Tipo	Descripción
IdSala	INTEGER	Clave primaria autoincrementable.
CodigoSala	TEXT	Código alfanumérico de 6 caracteres generado en el cliente. Debe ser único.
Nombre	TEXT	Nombre descriptivo de la sala.
IdHost	INTEGER	Clave foránea a Usuarios . Usuario que creó y administra la sala.
Estado	TEXT	Estado de la sala: Activa o Cerrada . Valor por defecto: Activa .
FechaCreacion	TEXT	Fecha y hora de creación en formato ISO 8601 (UTC).

Cuadro 7: Diccionario de datos – Salas

Tabla: SolicitudesSala

Campo	Tipo	Descripción
IdSolicitud	INTEGER	Clave primaria autoincrementable.
IdSala	INTEGER	Clave foránea a Salas . Sala a la que se solicita ingreso.
IdUsuario	INTEGER	Clave foránea a Usuarios . Usuario que solicita ingresar.
Estado	TEXT	Estado de la solicitud: Pendiente , Admitido o Rechazado .
FechaSolicitud	TEXT	Fecha y hora de la solicitud en formato ISO 8601.

Cuadro 8: Diccionario de datos – SolicitudesSala

10. Diagrama de Secuencia

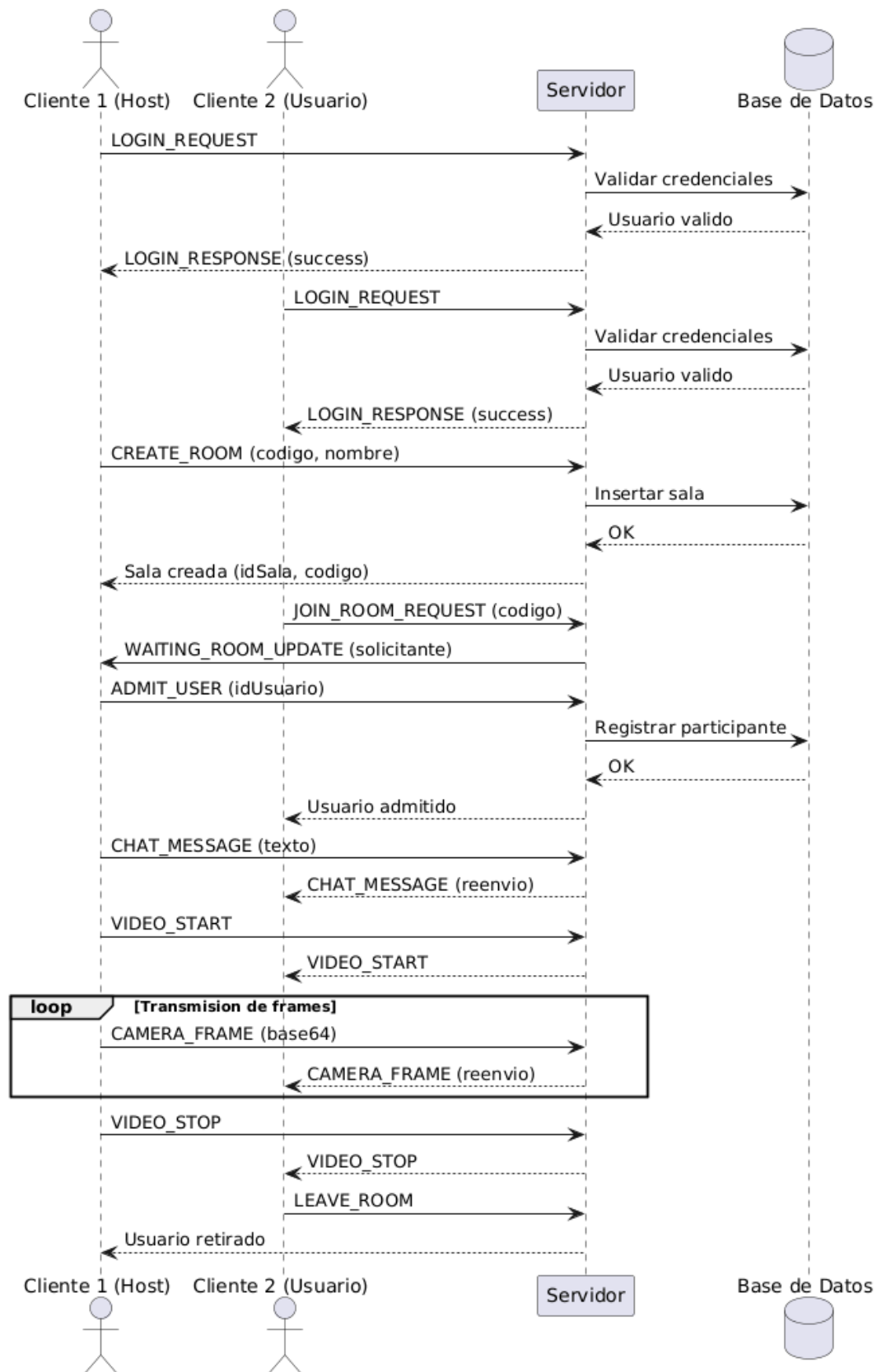


Figura 3: Diagrama de secuencia del flujo de la aplicación

11. Diagrama de Actividades

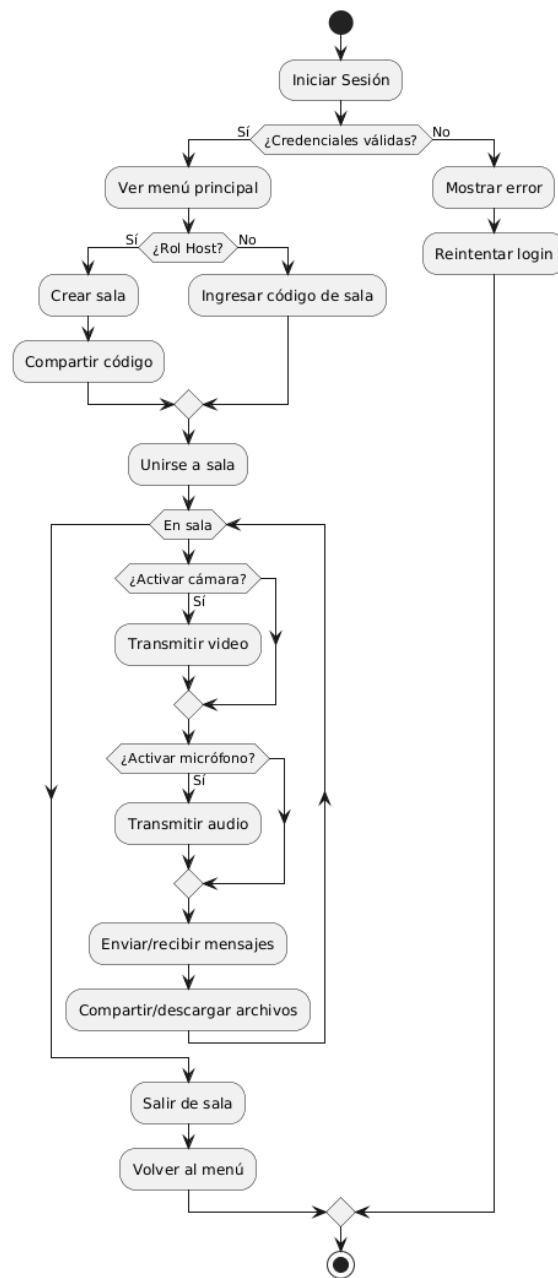


Figura 4: Diagrama de actividades del sistema

12. Diagrama de Clases

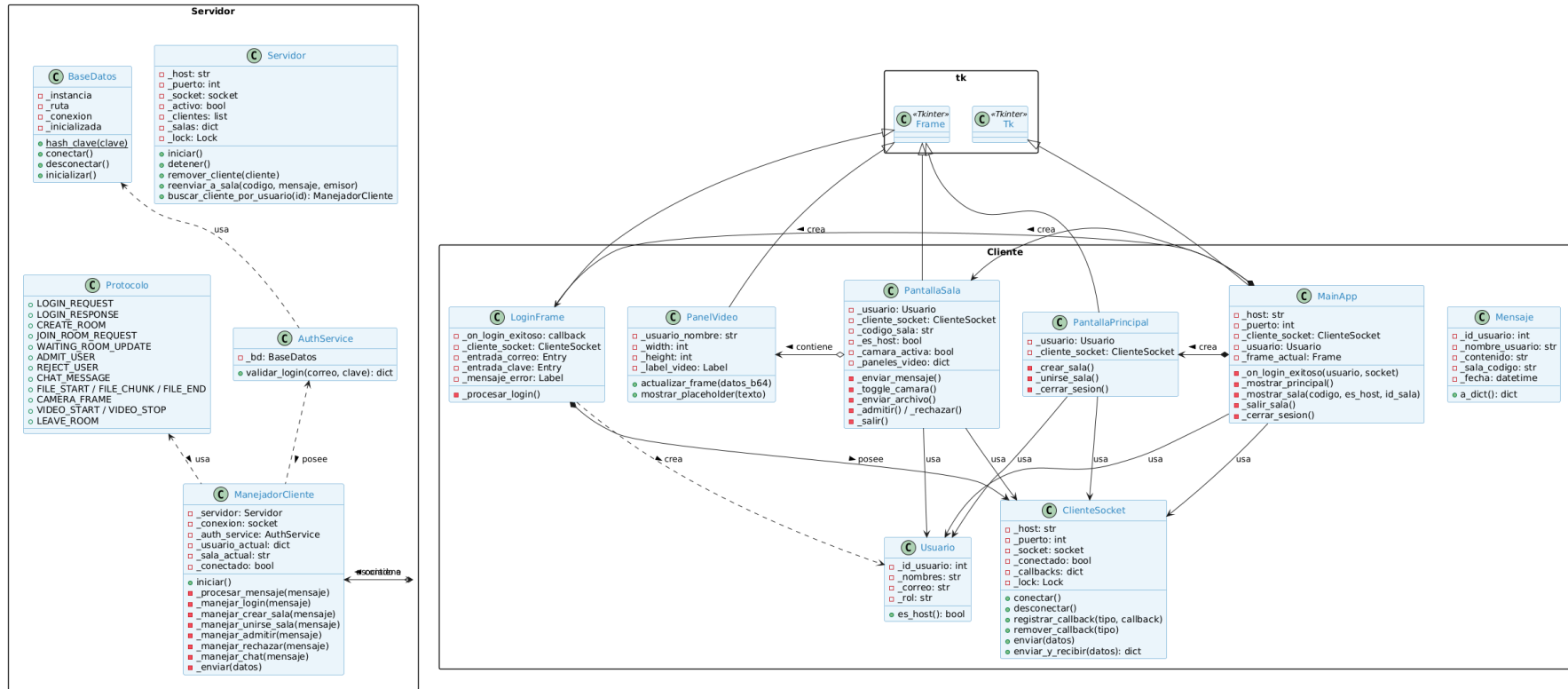


Figura 5: Diagrama de clases del sistema

13. Descripción del Código

Esta sección explica cómo funciona cada componente del sistema, detallando el rol de las clases y de qué forma se aplican los principios de la Programación Orientada a Objetos y los patrones de diseño para estructurar una aplicación modular, escalable y robusta.

13.1. Funcionamiento de las Clases y Componentes

El sistema está dividido en tres componentes principales: el Lado del Servidor, el Lado del Cliente y los Gestores de Hardware/Archivos.

13.1.1. Lado del Servidor

El servidor gestiona la persistencia de datos y la comunicación concurrente entre clientes:

- **Protocolo** (`Servidor/protocolo.py`): Define las constantes de mensaje que viajan serializadas en formato JSON sobre el socket TCP, sirviendo como diccionario del protocolo.
- **BaseDatos** (`Servidor/base_datos.py`): Inicializa y mantiene la base de datos SQLite. Centraliza las consultas, registros de salas y autenticaciones.
- **AuthService** (`Servidor/auth_service.py`): Valida las credenciales de los usuarios comparando el hash SHA-256 almacenado con el ingresado por el cliente.
- **ManejadorCliente** (`Servidor/manejador_cliente.py`): Representa el hilo asignado por el servidor para atender a un cliente. Recibe e interpreta los mensajes JSON delimitados por `\n` y despacha las peticiones.
- **Servidor** (`Servidor/servidor.py`): Clase principal que abre el socket TCP de escucha, acepta nuevas conexiones de clientes y crea un hilo daemon con su respectivo `ManejadorCliente`.

13.1.2. Lado del Cliente

El cliente proporciona la interfaz gráfica en Tkinter y procesa la interacción con el usuario y la red:

- **MainApp** (`Cliente/main.py`): Ventana raíz de Tkinter que administra la transición fluida entre las diferentes vistas de la aplicación.
- **ClienteSocket** (`Cliente/cliente_socket.py`): Abstrae el socket del cliente, manteniendo un hilo de escucha para procesar mensajes entrantes en tiempo real.
- **Modelos** (`Cliente/modelos/usuario.py`, `mensaje.py`): Objetos que modelan a un usuario conectado y los mensajes del chat, facilitando su transporte y manipulación.
- **Pantallas** (`LoginFrame`, `PantallaPrincipal`, `PantallaSala`, `PanelVideo`): Subclases de Tkinter especializadas en dibujar los formularios de login, el menú de salas, la sala de reuniones y las pantallas de video respectivamente.

13.1.3. Gestores de Red y Hardware

Clases auxiliares delegadas que encapsulan la complejidad de manejar hardware y transmisión:

- **GestorVideo** (`Cliente/gestores/gestor_video.py`): Controla la captura de la cámara web, decidiendo si usa OpenCV o FFmpeg, y envía los frames comprimidos y codificados en Base64.
- **GestorAudio** (`Cliente/gestores/gestor_audio.py`): Captura audio en PCM usando `sounddevice` y reproduce el audio recibido en un hilo y cola FIFO independientes.
- **GestorArchivos** (`Cliente/gestores/gestor_archivos.py`): Controla la transferencia segmentada de archivos en Base64, partiendo el archivo en trozos de 1500 bytes y reconstruyéndolo en el receptor.

13.2. Aplicación de los Pilares de la POO

Los pilares de la Programación Orientada a Objetos se aplican directamente en la estructuración de la aplicación para resolver problemas de modularidad y acoplamiento:

13.2.1. Encapsulamiento

Se utiliza para proteger el estado interno de los objetos, ocultando los sockets directos, colecciones de clientes o contraseñas, y exponiéndolas solo mediante métodos o propieda-

des controladas. Por ejemplo, en la clase `Usuario` (`Cliente/modelos/usuario.py`), los datos del perfil están protegidos con prefijos de guion bajo, y el acceso se restringe mediante decoradores `@property`. En el servidor, la lista de clientes conectados se protege detrás de un lock para evitar condiciones de carrera:

```
1 class Usuario:
2     def __init__(self, id_usuario, nombres, correo, rol):
3         self._id_usuario = id_usuario # Atributo privado
4         self._nombres = nombres
5         self._rol = rol
6
7     @property
8     def id_usuario(self):
9         return self._id_usuario
10
11     def es_host(self):
12         return self._rol.lower() == "host"
```

13.2.2. Herencia

Se implementa principalmente en la interfaz gráfica para reutilizar los componentes visuales de Tkinter. Clases como `MainApp` heredan de `tk.Tk`, y las pantallas (`LoginFrame`, `PantallaPrincipal`, `PantallaSala`, `PanelVideo`) heredan de `tk.Frame`, permitiendo extender y personalizar el comportamiento de contenedores gráficos estándar:

```
1 class PanelVideo(tk.Frame): # Hereda de tk.Frame
2     def __init__(self, master, usuario_nombre="", width=240, height=180,
3         bg="#222"):
4         super().__init__(master, bg=bg, width=width, height=height)
5         self.pack_propagate(False)
6         # Widgets especificos para dibujar el video
```

13.2.3. Polimorfismo

Permite tratar de forma homogénea diferentes pantallas dentro del flujo de la aplicación. En `MainApp`, la variable `_frame_actual` puede contener instancias de `LoginFrame`, `PantallaPrincipal` o `PantallaSala`. Al cambiar de pantalla, el contenedor llama polimórficamente a los métodos heredados como `pack()` y `destroy()`:


```
1 # Transición dinámica de pantallas en MainApp
2 self._frame_actual.destroy() # Destruye la pantalla previa
3 self._frame_actual = PantallaSala(self, ...) # Instancia nueva pantalla
4 self._frame_actual.pack(fill=tk.BOTH, expand=True) # Empaqueta
    polimórficamente
```

13.2.4. Abstracción

Oculta la complejidad técnica de la red y el hardware de las pantallas de usuario. `ClienteSocket` encapsula toda la lógica del socket TCP, serialización JSON y buffer, ofreciendo solo métodos de alto nivel como `enviar` o `registrar_callback`:

```
1 class ClienteSocket:
2     def conectar(self): ...
3     def enviar(self, datos):
4         self._socket.send((json.dumps(datos) + "\n").encode("utf-8"))
5     def registrar_callback(self, tipo, callback):
6         self._callbacks[tipo] = callback
```

13.3. Patrones de Diseño Implementados

Los patrones de diseño se utilizaron para dar una solución limpia y estandarizada a las problemáticas típicas de una aplicación cliente-servidor:

13.3.1. Singleton (Instancia Única)

Garantiza que la clase `BaseDatos` posea una única instancia en todo el ciclo del servidor, centralizando las conexiones y evitando fallos por concurrencia en SQLite:

```
1 class BaseDatos:
2     _instancia = None
3
4     def __new__(cls, *args, **kwargs):
5         if cls._instancia is None:
6             cls._instancia = super().__new__(cls)
7         return cls._instancia
```

13.3.2. Observer (Observador)

El `ClienteSocket` actúa como sujeto que recibe mensajes de la red, y las diferentes pantallas o gestores se registran como observadores a través de callbacks, recibiendo notificaciones cuando se detecta un mensaje que les compete:

```
1 # Suscripcion (Observadores se registran)
2 self._cliente_socket.registrar_callback("CHAT_MESSAGE", self._
  _recibir_chat)
3
4 # Notificacion en ClienteSocket
5 def _escuchar(self):
6     ...
7     cb = self._callbacks.get(mensaje.get("type"))
8     if cb:
9         cb(mensaje) # Notifica al callback registrado
```

13.3.3. Strategy (Estrategia)

El `GestorVideo` define algoritmos de captura de video intercambiables. Si el cliente dispone de OpenCV instalado en su sistema, el gestor utiliza `cv2.VideoCapture` para capturar los frames; en caso contrario, alterna a una estrategia basada en `ffmpeg` leyendo directamente desde el dispositivo de video local:

```
1 def _capturar_y_enviar(self):
2     if self._backend == "opencv":
3         self._capturar_con_opencv()
4     else:
5         self._capturar_con_ffmpeg()
```

13.3.4. Facade (Fachada)

El `ManejadorCliente` en el servidor funciona como una fachada que centraliza y simplifica el uso de múltiples subsistemas complejos del servidor (como `AuthService`, `BaseDatos` y la distribución de mensajes de `Servidor`), exponiendo solo una interfaz simple mediante el método `iniciar()`.

13.3.5. Command (Comando)

El protocolo de red encapsula las intenciones del usuario en objetos JSON que viajan por el socket con un campo `type` que actúa como comando. Esto permite enrutar y procesar de forma genérica las peticiones:

```
1 # Servidor interpreta y despacha comandos de forma desacoplada
2 if tipo == Protocolo.LOGIN_REQUEST:
3     self._manejar_login(mensaje)
4 elif tipo == Protocolo.CREATE_ROOM:
5     self._manejar_crear_sala(mensaje)
```

13.3.6. Factory Method (Método Fábrica)

MainApp delega la creación e instanciación de las pantallas visuales a métodos fábrica específicos (`_mostrar_login()`, `_mostrar_principal()`, `_mostrar_sala()`). El cliente no conoce cómo se construyen las pantallas, solo invoca la fábrica:

```
1 def _mostrar_sala(self, codigo_sala, es_host, id_sala):
2     self._frame_actual = PantallaSala(self, self._usuario, self.
3         _cliente_socket, codigo_sala, es_host, id_sala, self._salir_sala)
4     self._frame_actual.pack(fill=tk.BOTH, expand=True)
```

13.3.7. Template Method (Método Plantilla)

En la transmisión de video, la captura e inicialización siguen un esqueleto predefinido (establecer conexión, inicializar buffer, iterar captura, codificar, enviar y liberar recursos). La lógica del bucle de captura se define en la superclase del gestor, mientras que los pasos específicos para la lectura de la cámara cambian según la estrategia.

14. Protocolos y Normativas Utilizados

El sistema utiliza **TCP** como protocolo de transporte para todas las comunicaciones. Cada funcionalidad prepara el dato de forma distinta antes de enviarlo, pero todas comparten el mismo canal confiable.

- **Video:** la cámara es capturada por OpenCV (o ffmpeg). El frame se comprime en formato JPEG y se convierte a texto mediante Base64 para poder viajar dentro de

un mensaje JSON. Elegimos TCP para transportar esos frames entre máquinas — aunque en sistemas reales como Zoom se usa UDP para menor latencia.

- **Audio:** el micrófono entrega bloques de audio crudo (PCM). Esos bytes también se convierten a Base64 y se envían por TCP. El receptor los reproduce en orden mediante una cola interna.
- **Archivos:** el archivo se lee completo, se codifica en Base64 y se corta en pedazos pequeños. Se envían con un mensaje de inicio, varios fragmentos y uno de fin — así el receptor sabe cuándo rearmar el archivo original.
- **Mensajes de control** (login, crear sala, chat, etc.): viajan directamente como objetos JSON sobre TCP, delimitados por el carácter `\n`.

14.1. Tabla de Mensajes del Protocolo

Los mensajes se envían en JSON sobre TCP, delimitados por `\n`.

Tipo	Dirección	Campos
LOGIN_REQUEST	Cliente → Servidor	correo, clave
LOGIN_RESPONSE	Servidor → Cliente	status, idUsuario, nombres, rol
CREATE_ROOM	Cliente → Servidor	codigo, nombre, idHost
JOIN_ROOM_REQUEST	Cliente → Servidor	codigo, idUsuario
WAITING_ROOM_UPDATE	Servidor → Host	idSala, solicitanteId, solicitanteNombre
ADMIT_USER	Host → Servidor	idSala, idUsuario
REJECT_USER	Host → Servidor	idSala, idUsuario
CHAT_MESSAGE	Cliente → Sala	roomCode, userName, message
FILE_START	Cliente → Sala	roomCode, fileName, fileSize
FILE_CHUNK	Cliente → Sala	roomCode, fileName, data
FILE_END	Cliente → Sala	roomCode, fileName
CAMERA_FRAME	Cliente → Sala	roomCode, userName, data (base64)
VIDEO_START	Cliente → Sala	roomCode, userName
VIDEO_STOP	Cliente → Sala	roomCode, userName
LEAVE_ROOM	Cliente → Servidor	roomCode

Cuadro 9: Tipos de mensaje del protocolo

15. Manual de Instalación y Ejecución

15.1. Requisitos

- Python 3.10 o superior
- Pillow (instalar con `pip install -r requirements.txt`)
- Opcional: OpenCV (`pip install opencv-python`) o ffmpeg

15.2. Ejecución

1. Iniciar el servidor:

```
1 python -m Servidor.servidor
```

2. Iniciar uno o más clientes:

```
1 python -m Cliente.main
```

15.3. Datos de Prueba

Nombre	Correo	Clave	Rol
Luis Tasayco	luis@email.com	123456	Host
Ana Torres	ana@email.com	123456	Usuario
Carlos Ruiz	carlos@email.com	123456	Usuario

Cuadro 10: Usuarios de prueba

16. Conclusiones

- Se implementó un prototipo funcional de videoconferencia con arquitectura cliente-servidor.
- Los sockets TCP con protocolo JSON proporcionan una base sólida para comunicación en tiempo real.
- Los cuatro pilares de la POO – encapsulamiento, herencia, polimorfismo y abstracción – fueron aplicados consistentemente en todo el código.

- Se identificaron y aplicaron siete patrones de diseño: Singleton, Observer, Strategy, Facade, Command, Template Method y Factory Method.
- La POO permitió organizar el código en clases especializadas con responsabilidades claras.
- La concurrencia con hilos permite atender múltiples clientes simultáneamente.
- La transmisión de video se logró mediante captura, compresión JPEG y codificación base64.